Kim Martínez Taesoo
Zertuche Zamora Sergio Gabriel
Méndez Guerrero Pablo Daniel
Aguilar Gómez Lesly Dariana

Programación de Sistemas de Base I

Propuesta de Proyecto Integrador – PipeSpec

Definición del Proyecto

PipeSpec: Lenguaje Específico de Dominio (DSL) para Orquestación y Verificación Sintáctica de Pipelines CI/CD

El presente proyecto consiste en el diseño formal y la implementación del frontend de un compilador (específicamente sus etapas de análisis léxico y análisis sintáctico con generación de Árbol Sintáctico Abstracto - AST).

El compilador será programado íntegramente de forma nativa en Go (Golang). La elección de Go obedece a tres factores técnicos clave:

1. Rendimiento y portabilidad: Genera binarios ejecutables nativos sin dependencias externas ni sobrecarga de máquinas virtuales.
2. Tipado estático y modelado de datos: El sistema de tipos, estructuras e interfaces implícitas de Go permite definir contratos formales estrictos para cada nodo del AST (Node, Statement, Expression), facilitando un recorrido recursivo predecible y seguro en memoria.
3. Poder de parsing construido internamente: Go cuenta con librarías robustas como go/token, go/scanner, go/parser y go/ast. Aunque inicialmente utilizadas para parsear Go en sí, estas sirven como base para diseñar herramientas personalizadas.

Justificación

En flujos continuos de entrega y despliegue (CI/CD), la definición de infraestructura y pipelines depende casi exclusivamente de archivos de configuración serializados en YAML o JSON. Estos formatos carecen de una gramática formalmente tipada en tiempo de edición, provocando que errores como indentaciones erróneas, referencias circulares o propiedades invalidas solo se detecten tras haber iniciado la ejecución de un trabajo en la nube. PipeSpec soluciona este problema verificando la definición jerárquica obligatoria de pipelines y etapas, modelado nativo de dependencias concurrentes,

bloques de ejecución estrictos delimitados por niveles, y cláusulas de guarda lógica condicionadas.

Como lenguaje destino el proyecto se inclina hacía Ensamblador x86-64 con sintaxis NASM. Go ofrece un rendimiento nativo óptimo para recorrer árboles complejos en memoria y formatear cadenas de texto masivas para la generación del archivo .asm mediante strings.Builder y tipos estrictamente tipados.

Ejemplo de código fuente en PipeSpec

```
pipeline "deploy-service":
        stage "lint":
                image = "golang:1.24"
                exec:
                        "golangci-lint run"

        stage "test" depends_on ["lint"]:
                image = "golang:1.24"
                timeout = 300
                exec:
                        "go test -v ./..."

        stage "deploy" depends_on ["test"]:
                image = "alpine/k8s:1.30"
                timeout = 180
                when = [[ branch == "main" && dry_run == false ]]
                exec:
                        "kubectl apply -f deploy/"
```

Delimitación

El proyecto se concentrará estrictamente en la etapa de análisis sintáctico, donde se visualizará un Árbol Sintáctico Abstracto.

Tabla de símbolos

Categoría Léxica	Token	Lexema/Patrón	Descripción Sintáctica
Palabras Reservadas	PIPELINE	pipeline	Inicia la declaración global del flujo de CI/CD.
	STAGE	stage	Define una unidad de trabajo o etapa dentro del pipeline.
	DEPENDS_ON	depends_on	Cláusula opcional que establece el grafo de dependencias de una etapa.
	IMAGE	image	Especifica el contenedor o imagen de ejecución
	TIMEOUT	Timeout	Tiempo límite de ejecución en segundos
	WHEN	when	Guarda lógica condicional para ejecutar la etapa.
	EXEC	exec	Encabezado del bloque de comandos a ejecutar.
Literales	STRING	"[^"\n]*"	Cadenas de texto entrecomilladas (nombres, rutas, imágenes).
	INTEGER	[0-9]+	Valores numéricos enteros positivos (duraciones).
	BOOLEAN	true | false	Literales lógicos de estado para guardas.
Identificadores	IDENT	[a-zA-Z_][a-zA-Z0-9]*	Variables de entorno o contexto evaluadas (e.g. branch, dry_run).
Operadores	ASSING	=	Operador de asignación clave-valor en propiedades.
	EQ	==	Comparación relacional de igualdad en guardas.

	AND	&&	Conjunción lógica en expresiones condicionales.
	OR	||	Disyunción lógica en expresiones condicionales.
Delimitadores	COLON	:	Delimitador de apertura de bloque tras directivas.
	LBRACKET	[	Apertura de lista de dependencias.
	RBRACKET	]	Cierre de lista de dependencias.
	DOUBLE_LBRACKET	[[	Apertura estricta de guarda de control.
	DOUBLE_RBRACKET	]]	Cierre estricto de guarda de control.
	COMMA	,	Separador de elementos en listas de dependencias.
Control Virtual	NEWLINE	\n	Final formal de sentencia lógica.
	INDENT	Virtual	Incremento en la pila de indentación de bloque.
	DEDENT	Virtual	Decremento en la pila de indentación de bloque.